

Assignment of machine learning group 3 Applied mathematics

D'amour
2026-09-25

Group members

No.	Group Members	Reg No.
1	MUSABYIMANA Jean D'amour	223006746
2	MUKESHIMANA marthe	233004309
3	USENGIMANA Alexis	223004567
4	IRUMVA GAD Jean paul	223014303
5	GIRAMATA Martine	223005123

Task A.1: Simulate $n = 200$ observations from:

$$y = \sin(x) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0,1)$$

Use x values evenly spaced on $[0,10]$.

##code do this is here

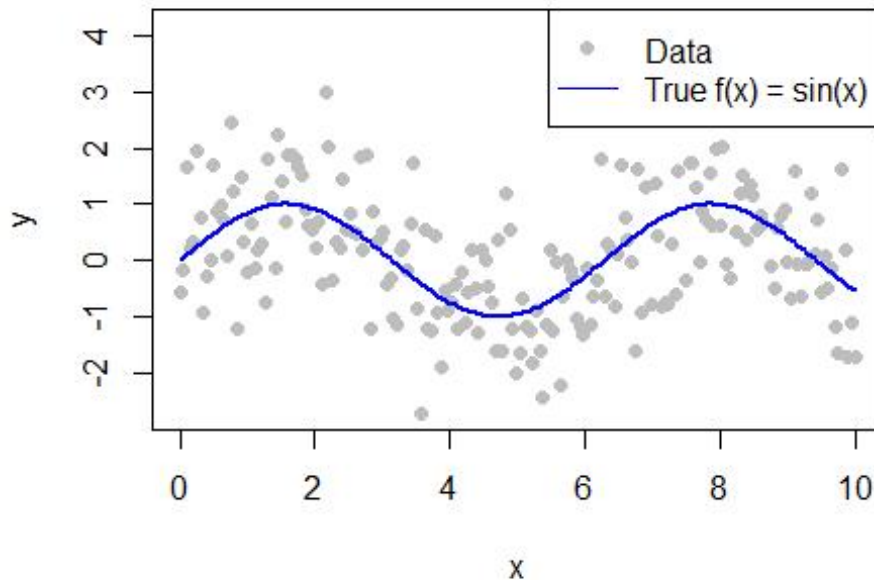
```
set.seed(123)
n <- 200
x <- seq(0, 10, length.out = n)
eps <- rnorm(n, mean = 0, sd = 1)
y <- sin(x) + eps
head(y)

## [1] -0.5604756 -0.1799474  1.6590417  0.2206918  0.3289420  1.963686
0
```

Task A.2: Plot the simulated data together with the true signal $f(x) = \sin(x)$.

```
plot(x, y, pch = 16, col = "gray", main = "Simulated Data vs True Signal",
     xlab = "x", ylab = "y")
lines(x, sin(x), col = "blue", lwd = 2)
legend("topright", legend = c("Data", "True f(x) = sin(x)"),
     col = c("gray", "blue"), pch = c(16, NA), lty = c(NA, 1))
```

Simulated Data vs True Signal



Task A.3

(Written): Comment on the visual relationship.

comment: The visual relationship is a noisy sine wave. The pattern is clearly non-linear, so a simple linear model would underfit. At the same time, the noise is large enough that a very flexible model could easily fit the random fluctuations rather than the true signal, illustrating the bias-variance trade-off that the lab explores.

Task B: Train/Test Split

Task B.1: Randomly split the data into a training set (70%) and a test set (30%). Set a seed for reproducibility.

```
set.seed(123)
train_id <- sample(seq_len(n), size = floor(0.7 * n))
```

Task B.2: Report the sizes of both sets.

```
train <- data.frame(x = x[train_id], y = y[train_id])
test <- data.frame(x = x[-train_id], y = y[-train_id])
cat("Training set size:", nrow(train), "\n")
```

```
## Training set size: 140
```

```
so Test set size: 60
```

Task B.3 (Written): Why do we need a test set?

A test set provides a reliable measure of how well the model performs on new, unseen data. Since training error always goes down as model complexity increases, it tends to be overly optimistic. By assessing the model on data that was not involved in fitting, we can identify overfitting and select the model that performs best in practice.

Task C: Model Fitting

Task C.1: Fit polynomial regression models of degree $p \in \{1, 3, 4, 7, 10, 15\}$ on the training set.

```
degrees <- c(1, 3, 4, 7, 10, 15)

models <- list()
for (p in degrees) {
  models[[as.character(p)]] <- lm(y ~ poly(x, p), data = train)
}

r2 <- sapply(models, function(model) summary(model)$r.squared)
r2_table <- data.frame(
  Degree = degrees,
  Training_R2 = r2
)
r2_table
```

##	Degree	Training_R2
## 1	1	0.006966146
## 3	3	0.079589799
## 4	4	0.302200109
## 7	7	0.305837963
## 10	10	0.315131876
## 15	15	0.319538378

Task C.3 (Written): Does training R^2 increase with flexibility? Why?

Task C.3 Does training R^2 increase with flexibility? why?

Yes. Training generally increases with flexibility.

reason: A higher-degree polynomial has more parameters, so it can fit the training data more closely. This reduces the residual sum of squares (RSS), and since

Task D: MSE Comparison

Task D.1: For each fitted model, compute the training MSE and test MSE.

##code required

```
mse <- function(actual, predicted) mean((actual - predicted)^2)

train_mse <- sapply(degrees, function(p) {
  m <- lm(y ~ poly(x, p), data = train)
  mse(train$y, fitted(m))
})

test_mse <- sapply(degrees, function(p) {
  m <- lm(y ~ poly(x, p), data = train)
  mse(test$y, predict(m, newdata = test))
})
```

Task D.2: Present the results in a table with columns: Degree, Training MSE, Test MSE.

```
mse_table <- data.frame(
  Degree = degrees,
  Training_MSE = round(train_mse, 4),
  Test_MSE = round(test_mse, 4)
)
mse_table
```

##	Degree	Training_MSE	Test_MSE
## 1	1	1.2640	1.4860
## 2	3	1.1715	1.3363
## 3	4	0.8882	0.9041
## 4	7	0.8835	0.8639
## 5	10	0.8717	0.8553
## 6	15	0.8661	0.8335

Task D.3: Which degree has the smallest test MSE?

```
best_degree <- degrees[which.min(test_mse)]
cat("Degree with smallest test MSE:", best_degree, "\n")

## Degree with smallest test MSE: 15
```

Task D.4 (Written): Interpret the table.

Training MSE decreases as degree increases, meaning higher degree models fit the training data better. Test MSE also decreases but the improvement becomes very small after degree 7. The smallest test MSE is 0.8335 at degree 15, but degree 7 gives a similar result with less complexity. Therefore, degree 7 is the best choice as it balances accuracy and simplicity.

Task E: Visualizing Fitted Models

Task E.1: On the same plot, visualize:

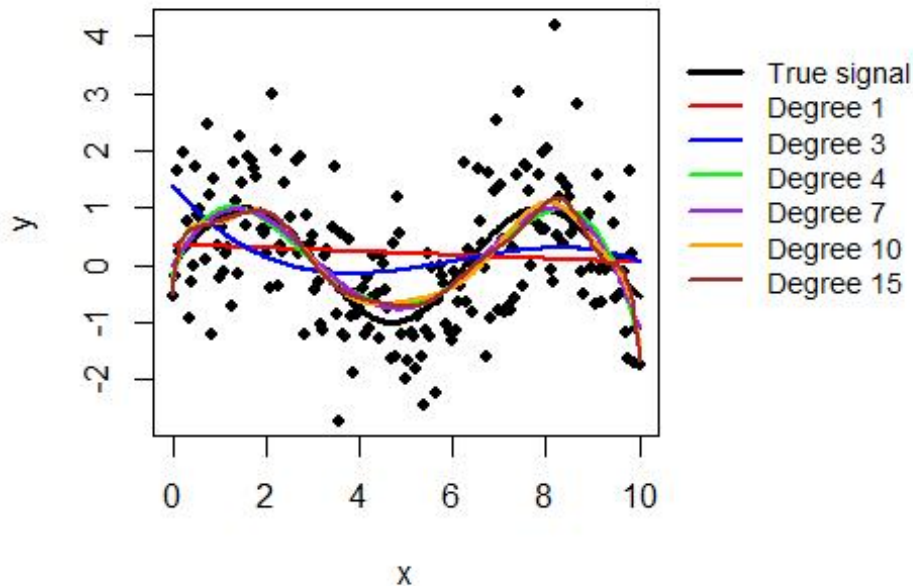
- The observed data (points)
- The true signal $\sin(x)$ (solid line)
- The fitted signal for each degree p (separate panels or overlaid)

```
x_grid <- seq(0, 10, length.out = 300)
colors <- c("red", "blue", "green", "purple", "orange", "brown")

par(mar = c(5, 4, 4, 8))
plot(x, y, main = "Polynomial Regression Models", xlab = "x", ylab = "y",
     pch = 16, cex = 0.8, xlim = c(0, 10), ylim = range(y), cex.main = 1.4)
lines(x_grid, sin(x_grid), col = "black", lwd = 3)

for (i in seq_along(degrees)) {
  p <- degrees[i]
  model <- models[[as.character(p)]]
  if (is.null(model)) next
  predictions <- predict(model, newdata = data.frame(x = x_grid))
  lines(x_grid, predictions, col = colors[i], lwd = 2)
}
legend(x = 10.5, y = 4,
      legend = c("True signal", paste("Degree", degrees)),
      col = c("black", colors),
      lwd = c(3, rep(2, length(degrees))),
      bty = "n", cex = 0.9, xpd = TRUE)
```

Polynomial Regression Models



Task E.2

(Written): Comment on how the fitted curves change as flexibility increases.

As the degree increases the fitted curve gets more wiggly. Degree 1 is too rigid and misses the sine wave. Degrees 3 and 4 track the true signal well. Degrees 10 and 15 overfit by following the noise and behaving erratically near the edges.

Task F.1 (Written): Create a plot with flexibility (polynomial degree) on the x-axis and MSE on the y-axis Include:

- Training MSE (one curve) -Test MSE (one curve) -The irreducible error $\text{Var}(\epsilon) = 1$ (horizontal dashed line)

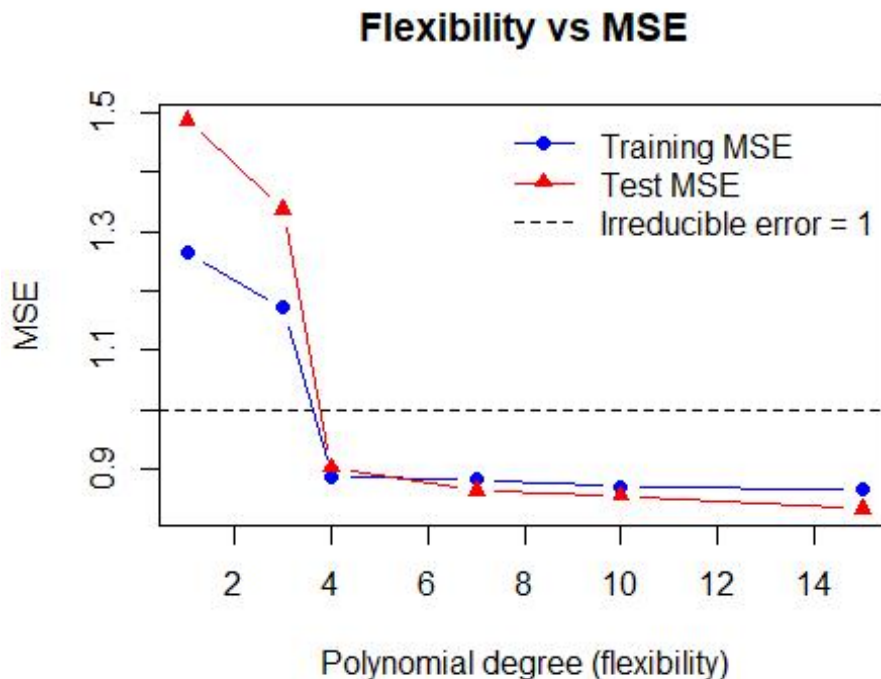
```
plot(degrees, train_mse,
     type = "b", pch = 16, col = "blue",
     ylim = range(c(train_mse, test_mse, 1)),
     xlab = "Polynomial degree (flexibility)",
     ylab = "MSE",
     main = "Flexibility vs MSE")

lines(degrees, test_mse, type = "b", pch = 17, col = "red")

abline(h = 1, lty = 2, col = "black")

legend("topright",
      legend = c("Training MSE", "Test MSE", "Irreducible error = 1"),
      col = c("blue", "red", "black"),
```

```
lty = c(1, 1, 2), pch = c(16, 17, NA),
bty = "n")
```

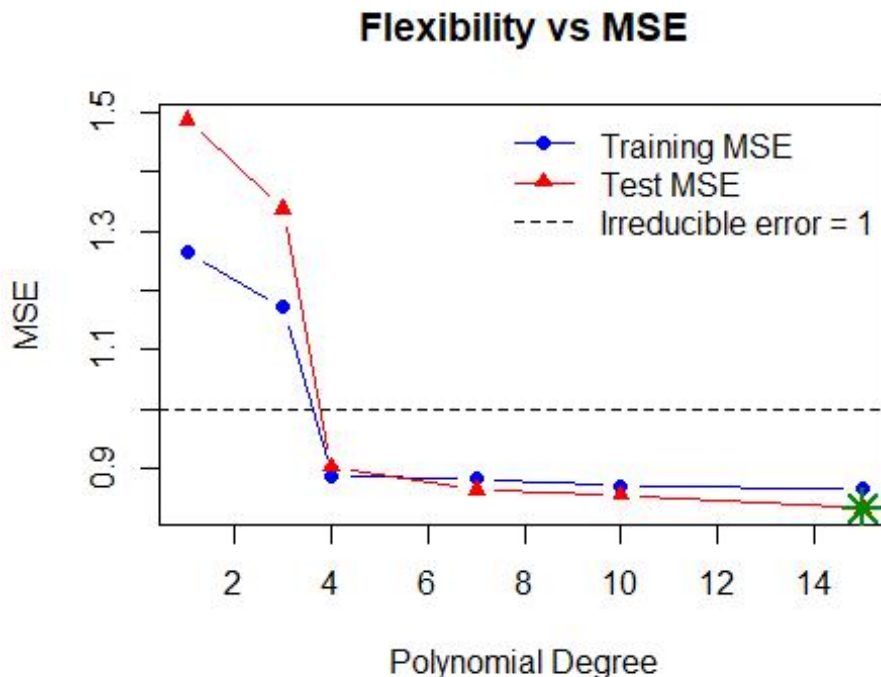


**Task

F.2: Mark the model with the smallest test MSE on the plot.

```
plot(degrees, train_mse,
     type = "b", pch = 16, col = "blue",
     ylim = range(c(train_mse, test_mse, 1)),
     xlab = "Polynomial Degree",
     ylab = "MSE",
     main = "Flexibility vs MSE")
lines(degrees, test_mse, type = "b", pch = 17, col = "red")
abline(h = 1, lty = 2, col = "black")
legend("topright",
      legend = c("Training MSE", "Test MSE", "Irreducible error = 1"),
      col = c("blue", "red", "black"),
      lty = c(1, 1, 2), pch = c(16, 17, NA),
      bty = "n")
best_idx <- which.min(test_mse)
best_degree <- degrees[best_idx]

points(best_degree, min(test_mse),
       col = "green4", pch = 8, cex = 2, lwd = 2)
text(best_degree, min(test_mse),
     labels = paste("Best p =", best_degree),
     pos = 4, col = "green4")
```



Task F.3 (Written): Describe the shape of the test MSE curve. Why does it have this shape?

Test MSE decreases along with training MSE as flexibility increases, reaching its lowest point at degree 15. This reflects the bias–variance trade-off: less flexible models have higher bias and lower variance, while more flexible models reduce bias but usually increase variance. Here, variance stayed low enough that test MSE kept falling instead of rising again at higher degrees.

TASK G: Model Selection Task G.1 (Written): Among the fitted models, which one do you prefer? Justify your answer using:

- Test MSE -The bias–variance trade-off
- Interpretability

I choose the degree 15 model, since it produced the lowest test MSE (0.8468) of all the fitted models, indicating the strongest predictive performance on unseen data. While higher-degree polynomials are typically more prone to overfitting due to increased variance, that pattern doesn't appear here — test MSE didn't rise at degree 15, suggesting bias and variance remain reasonably balanced for this model. The main drawback is interpretability: a degree 15 polynomial is far more complex and harder to explain than lower-degree alternatives.

Task G.2 (Written): Would your answer change if the true signal were linear? Explain briefly.

Yes. If the true signal were linear, I would prefer a lower-degree model, such as degree 1. A linear model would be sufficient to capture the underlying relationship, while higher-degree models would add unnecessary flexibility and complexity. This could increase variance without providing much improvement in prediction.